Solution: Maximum Length of a Concatenated String with Unique Characters

Problem Statement

Given an array `arr` containing strings, return the `maximum length` of the string that can be formed by the concatenations of the `subsequence` of `arr`, having unique characters.

A `subsequence` is an array that can be formed from `arr` by deleting 0 or more elements without changing the order of the remaining elements.

Examples

- **Example 1**:
  - **Input**: ["ab", "cd", "e"]
  - **Expected Output**: 5
  - **Justification**: Concatenating "ab", "cd", and "e" forms "abcde" which has all unique characters and is of maximum length 5.
- **Example 2**:
  - **Input**: ["a", "abc", "d", "de", "def"]
  - **xpected Output**: 6
  - **Justification**: Concatenating "abc", and "def" forms "abcdef" which has all unique characters and is of maximum length 6.
- **Example 3**:
  - **Input**: ["xyz", "y", "z"]
  - **Expected Output**: 3
  - **Justification**: We can choose only "xyz" string. Hence, the maximum length is 3.

Solution

The approach to solving the maximum length of a concatenated string with unique characters from a list involves an iterative method that expands on unique combinations progressively. Starting with an empty string, the algorithm combines each input string to previously validated unique sequences, updating a running maximum length whenever a longer unique combination is discovered.

This strategy ensures all possible combinations are explored efficiently, leveraging the simplicity of iterative expansion and the effectiveness of uniqueness checks. By focusing on incremental growth and meticulous validation, the method efficiently navigates through the potential combinations, identifying the optimal sequence length with precision and minimal computational overhead.

Step-by-step Algorithm

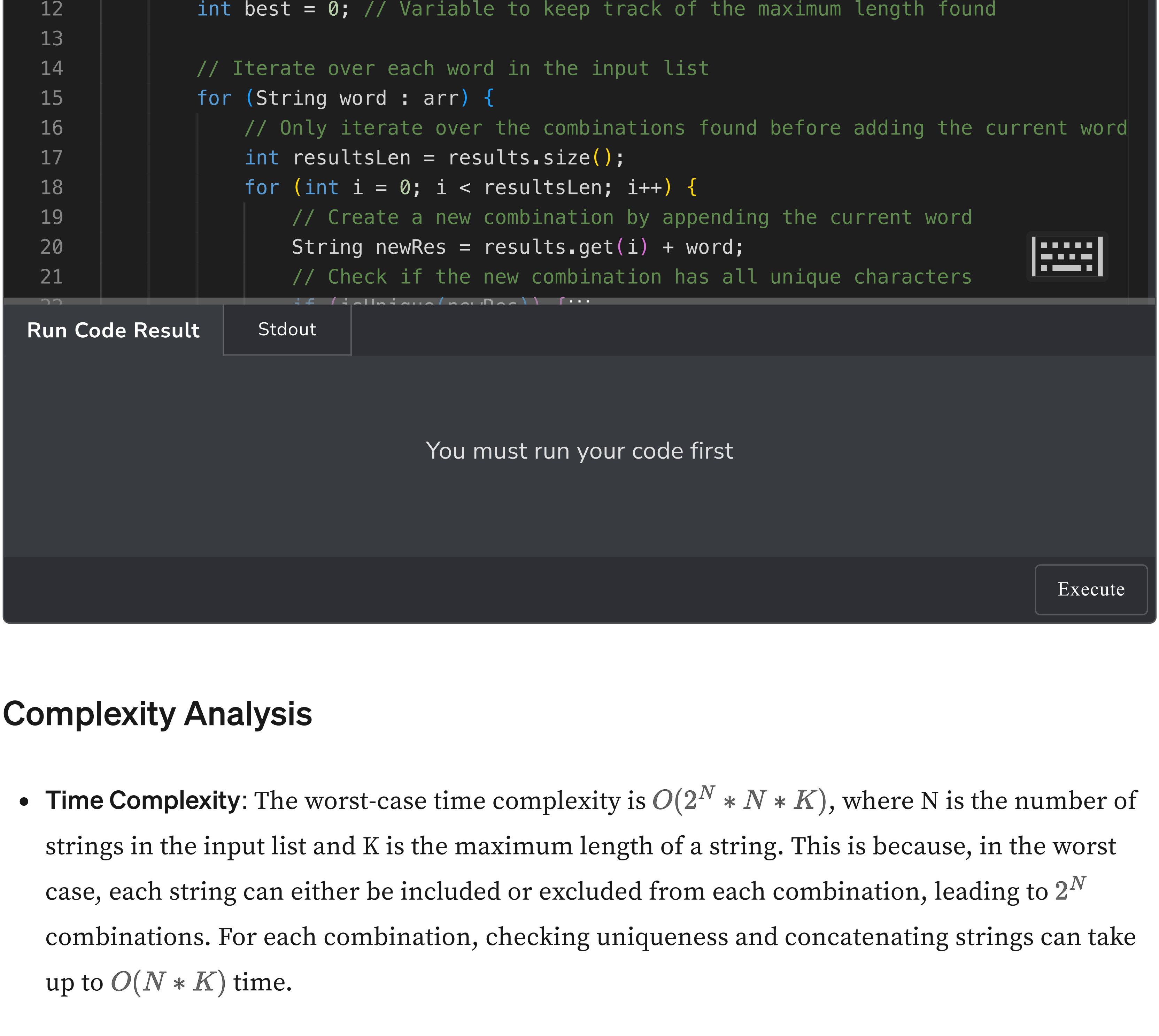
1. Initialize a list `results` containing one empty string. This list will store all unique character combinations found so far, starting with a possibility to build from an empty combination.
2. Initialize a variable `best` to 0, which will keep track of the maximum length of any unique character string formed throughout the process.
3. Iterate over each word in the given list `arr`.
   - For each `word` in `arr`, iterate over the combinations already in `results` (only those that existed before this iteration started, tracked by `resultsLen`).
     - Concatenate the current `word` with each existing combination (referred to as `existingCombination`).
     - Check if the new combination `newRes` (the result of concatenation) has all unique characters by calling the `isUnique` method.
       - If `newRes` is unique, add it to `results` for consideration in future combinations.
       - Update `best` to be the maximum of its current value or the length of `newRes`.
4. After exploring all words and their possible unique combinations, return `best` as the solution, representing the maximum length of a concatenated string with all unique characters.

Algorithm Walkthrough

Let's consider the Input `["a", "abc", "d", "de", "def"]`

- Initialize `results` with an empty string: `results = [""]`
- Initialize `best = 0`
- Start iterating over the array:
  - **First iteration with "a"**:
    - `results` contains `[""]`.
    - Concatenate "a" with "", resulting in "a".
    - Check uniqueness of "a", which is true.
    - Add "a" to `results`, now `results = ["", "a"]`.
    - Update `best` to 1.
  - **Second iteration with "abc"**:
    - Now, `results` contains `["", "a"]`.
    - Concatenate "abc" with "", resulting in "abc". Add "abc" to `results`, and update `best` to 3.
    - Concatenate "abc" with "a", but "aabc" is not unique, so skip adding.
    - `results` is now `["", "a", "abc"]`.
  - **Third iteration with "d"**:
    - `results` contains `["", "a", "abc"]`.
    - Concatenate "d" with each combination in `results`, resulting in valid unique strings "d", "ad", "abcd".
    - Add all valid combinations to `results`, now `results = ["", "a", "abc", "d", "ad", "abcd"]`.
    - Update `best` to 4 (length of "abcd"). Certainly, let's correct the walkthrough for the 4th and 5th iterations, focusing on accurately reflecting how the algorithm builds up to the correct maximum length with the input `["a", "abc", "d", "de", "def"]`.
  - **Fourth iteration with "de"**:
    - At this point, `results` contains `["", "a", "abc", "d", "ad", "abcd"]`.
    - Concatenate "de" with each valid combination from the previous step:
    - Concatenating "de" with "", "a", and "d" results in "de", "ade", and "dde", respectively. However, "dde" is not unique and is not added.
    - Concatenating "de" with "abc" results in "abcde", which is unique and added to `results`.
    - "de" with "ad" forms "adde", which is not unique and thus not added.
    - "de" with "abcd" would repeat characters and is thus not considered a valid new addition.
    - After this iteration, valid new combinations like "de", "ade", and "abcde" are added. **The most significant update here is "abcde"**, which increases `best` to 5.
    - `results` now includes these new unique combinations, making it larger, but for brevity, the key addition to note is `"abcde"`.
    - **Final `results` array at this point:** ["", "a", "abc", "d", "ad", "abcd", "de", "ade", "abcde", ...]
  - **Fifth iteration with "def"**:
    - Starting with the updated `results` from the last step, which now includes "abcde" among others.
    - Concatenate "def" with each valid combination from `results`:
    - Adding "def" to "", "a", "d", "ad", and notably "abcde" among others.
    - Most combinations either remain unchanged in terms of their contribution to `best` or are invalid due to repeated characters.
    - However, adding "def" to "abc" results in "abcdef", which is unique and thus a crucial addition.
    - The addition of "abcdef" is significant as it updates `best` to 6, which represents the longest unique combination possible from the given input.
  - **Final `results` array at this point:** ["", "a", "abc", "d", "ad", "abcd", "de", "ade", "abcde", "def", "adef", "abcdef", ...]

Code



```java
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class Solution {
    // Method to calculate the maximum length of a unique character string from a list
    public int maxLength(List<String> arr) {
        // Initialize a list to keep all combinations starting with an empty string
        List<String> results = new ArrayList<>();
        results.add(""); // Start with an empty string to build upon
        int best = 0; // Variable to keep track of the maximum length found

        // Iterate over each word in the input list
        for (String word : arr) {
            // Only iterate over the combinations found before adding the current word
            int resultsLen = results.size();
            for (int i = 0; i < resultsLen; i++) {
                // Create a new combination by appending the current word
                String newRes = results.get(i) + word;
                // Check if the new combination has all unique characters
```

You must run your code first

Complexity Analysis

- **Time Complexity**: The worst-case time complexity is $O(2^N * N * K)$, where N is the number of strings in the input list and K is the maximum length of a string. This is because, in the worst case, each string can either be included or excluded from each combination, leading to 2^N combinations. For each combination, checking uniqueness and concatenating strings can take up to $O(N * K)$ time.
- **Space Complexity**: The space complexity is $O(2^N * K)$, primarily due to storing all possible combinations of strings. In the worst case, this could include all 2^N combinations, each of up to K characters in length.

← Previous
Maximum Length Of A Concatenated String With Unique Characters (Medium)

Next →
Inorder Successor In BST (Medium)

☐ Mark as Completed